10Alytics

ACCREDITED BY THE
AMERICAN COUNCIL OF
TRAINING AND DEVELOPMENT

Automating the journey from code to production

Kubernetes Basics

Facilitator Ojetokun Victor

DEVOPS ENGINEERING - *WEEK 12*

KUBERNETES BASICS

Learning Objectives

- What is Kubernetes
- Kubernetes Architecture
- The Declarative Model
- Pods
- Deployments
- Services
- Key features and Use cases for Kubernetes

10Alytics | ACCREDITED BY THE AMERICAN COUNCIL OF TRAINING AND DEVELOPMENT

Kubernetes : The Orchestrator

Kubernetes (K8s) is an open-source **container orchestration platform** originally developed by Google and now maintained as part of the **Cloud Native Computing Foundation (CNCF)**.

Kubernetes helps you **deploy, manage, scale, and maintain containerized applications** across multiple machines.

Think of Kubernetes as a Manager for Containers

Instead of manually managing containers on different servers, Kubernetes manages them for you.

A **Kubernetes cluster** is a group of machines called **nodes** that work together to run containerized applications. Kubernetes makes it possible to manage these machines and applications as one coordinated system.

Key Features of Kubernetes

1. Self-Healing

Kubernetes continuously monitors the applications running in the cluster.

If a **Pod** crashes or stops working, Kubernetes can automatically restart or replace it to maintain the desired number of running instances.

2. Auto-Scaling

Kubernetes can automatically increase or decrease the number of Pods based on resource usage or other defined metrics.

For example, the **Horizontal Pod Autoscaler (HPA)** can scale an application based on:

- CPU utilization
- Memory utilization
- Custom or application metrics

Kubernetes : The Orchestrator

3. Rolling Updates

Kubernetes can update an application gradually instead of stopping all existing Pods at once.

During a **rolling update**, old Pods are progressively replaced with new Pods.

4. Intelligent Scheduling

Kubernetes has a component called the **Scheduler.**

The Scheduler decides which node should run a Pod based on factors such as:

- Available CPU and memory
- Resource requests and limits
- Node constraints
- Affinity and anti-affinity rules
- Taints and tolerations

Instead of manually deciding where every container should run, Kubernetes makes the scheduling decision based on the rules you provide.

5. Service Discovery and Load Balancing

Kubernetes **Services** provide a stable way for applications to communicate with Pods.

A Service can distribute network traffic across multiple healthy Pods running the same application.

Container Orchestration Problem

The Container Orchestration Problem

Docker makes it easy to **build, package, and run containers**, but managing containers at scale becomes difficult.

Running a few containers manually is manageable. But imagine running **1,000 containers across 50 servers:** manual management quickly becomes inefficient, error-prone, and difficult to maintain.

Limitations of Managing Docker Manually

- **No Automatic Self-Healing:** A crashed container may remain down unless someone restarts it.
- **Manual Scaling:** Increasing capacity requires manually starting additional containers.
- **Deployment Challenges:** Updating hundreds of containers can cause service disruption if not carefully managed.
- **Traffic Management:** Docker alone does not provide cluster-wide service discovery and load balancing.
- **Resource Management:** Containers may be poorly distributed, leaving some servers overloaded while others remain underutilized.

The Solution: Container Orchestration

As containerized applications grew larger, organizations needed a system that could **automatically deploy, scale, schedule, monitor, and recover containers across multiple servers**.

This need led to container orchestration platforms such as Kubernetes, Docker runs containers. Kubernetes manages containers at scale.

10Alytics | ACTD ACCREDITED BY THE AMERICAN COUNCIL OF TRAINING AND DEVELOPMENT

Docker Compose Vs Kubernetes

Area	Docker	Kubernetes
Main Purpose	Builds and runs containers	Manages and orchestrates containers at scale
Container Management	Manages containers on individual machines	Manages containers across multiple machines/nodes
Scaling	Usually requires manual configuration or additional tools	Supports automatic and manual scaling
Self-Healing	Can restart containers with configured restart policies	Automatically maintains the desired number of Pods
Deployment	Containers can be updated manually	Supports controlled deployments such as rolling updates
Scheduling	You generally decide where containers run	Scheduler determines suitable nodes for Pods
Load Balancing	Requires additional configuration/tools	Kubernetes Services provide service discovery and traffic distribution
Networking	Provides container networking	Provides cluster-level networking and Services
Resource Management	Basic container resource controls	Schedules workloads based on CPU, memory, and other constraints
High Availability	Requires additional setup	Designed to support highly available workloads
Desired State	Primarily command-based management	Declarative: you define the desired state
Best For	Building, packaging, and running containers	Managing containerized applications across a cluster

10Alytics | ACCREDITED BY THE AMERICAN COUNCIL OF TRAINING AND DEVELOPMENT

Kubernetes Architecture

A Kubernetes cluster consists of two main areas:

- **Control Plane –** Makes decisions and manages the overall cluster.
- **Worker Nodes –** Run the application's Pods and containers.

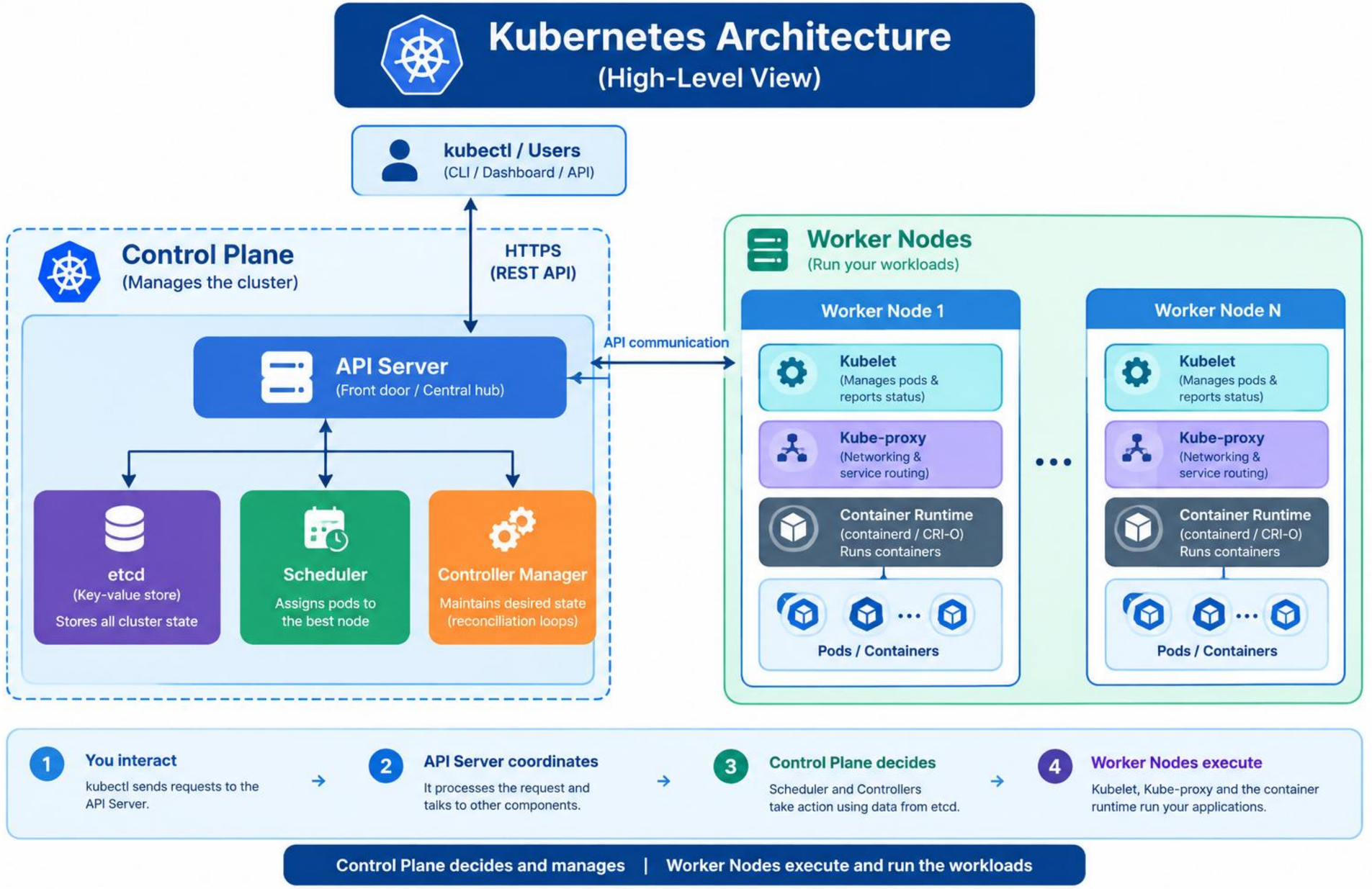
The **Kubernetes API Server** acts as the central communication point between users, the Control Plane, and Worker Nodes.

Control Plane Components

- **API Server:** The entry point to the cluster. Receives and processes Kubernetes requests.
- **etcd:** Distributed key-value database that stores the cluster's configuration and state.
- **Scheduler:** Determines which Worker Node should run newly created Pods.
- **Controller Manager:** Continuously compares the desired state with the actual state and takes corrective action.

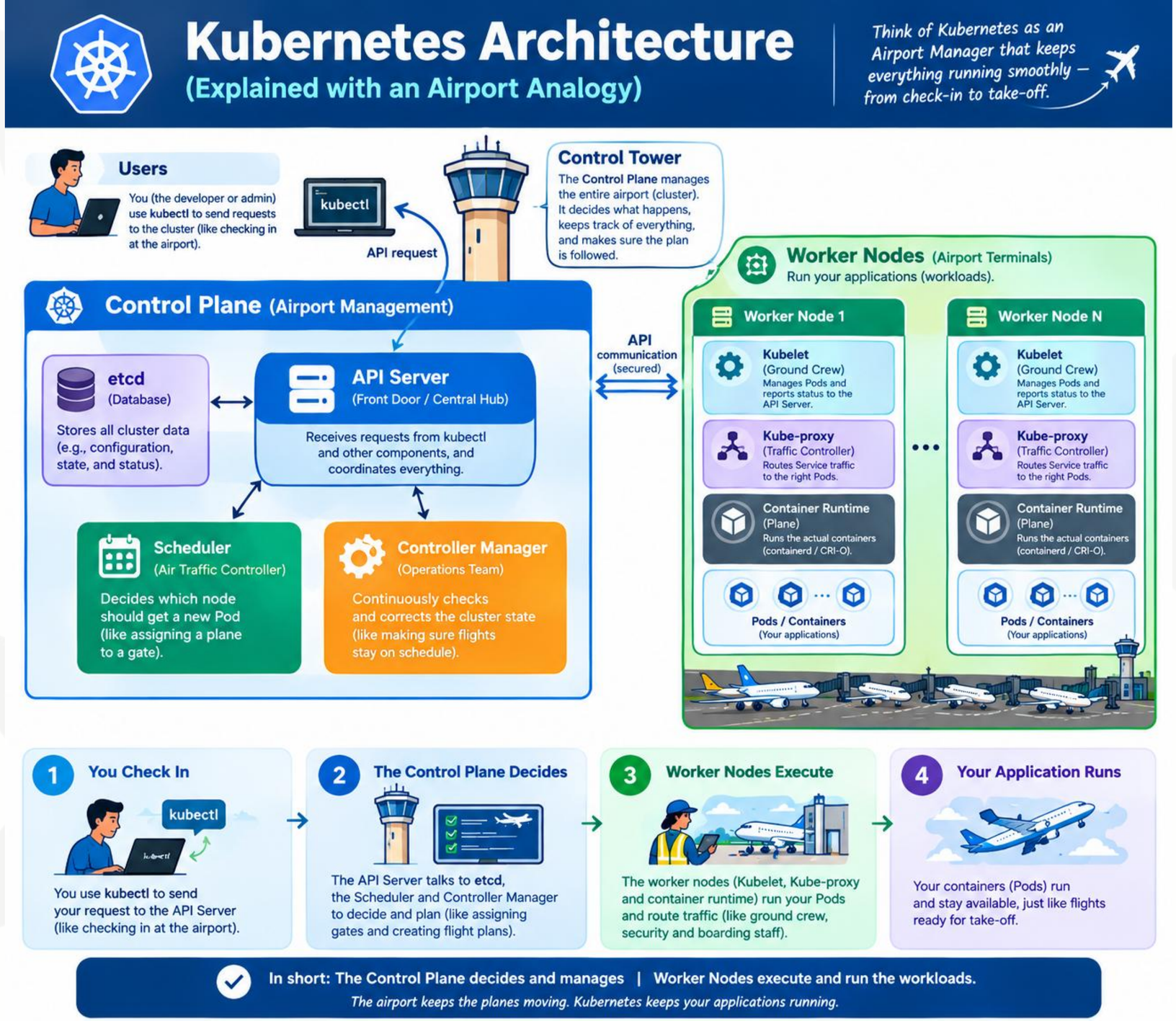
Worker Node Components

- **Kubelet:** Agent running on each Worker Node that manages Pods and communicates their status to the API Server.
- **Kube-proxy:** Helps implement Kubernetes Service networking and routes traffic to the appropriate Pods.
- **Container Runtime:** Software such as **containerd** or **CRI-O** that actually runs the containers.


Kubernetes Architecture
(High-Level View)
kubectl / Users
(CLI / Dashboard / API)
HTTPS
(REST API)
Control Plane
(Manages the cluster)
API Server
(Front door / Central hub)
API communication
etcd
(Key-value store)
Stores all cluster state
Scheduler
Assigns pods to the best node
Controller Manager
Maintains desired state (reconciliation loops)
Worker Nodes
(Run your workloads)
Worker Node 1
Kubelet
(Manages pods & reports status)
Kube-proxy
(Networking & service routing)
Container Runtime
(containerd / CRI-O)
Runs containers
Pods / Containers
Worker Node N
Kubelet
(Manages pods & reports status)
Kube-proxy
(Networking & service routing)
Container Runtime
(containerd / CRI-O)
Runs containers
Pods / Containers
1 You interact
kubectl sends requests to the API Server.
2 API Server coordinates
It processes the request and talks to other components.
3 Control Plane decides
Scheduler and Controllers take action using data from etcd.
4 Worker Nodes execute
Kubelet, Kube-proxy and the container runtime run your applications.
Control Plane decides and manages | Worker Nodes execute and run the workloads

10Alytics
ACCREDITED BY THE AMERICAN COUNCIL OF TRAINING AND DEVELOPMENT

Picture Explainer



The Declarative Model — How K8s works

Kubernetes is declarative, not imperative.

Instead of telling Kubernetes **how to perform each action**, you declare the **desired state**, and Kubernetes continuously works to make the actual environment match it.

The Control Loop — Reconciliation

Desired State

↓

Observe Actual State

↓

Compare / Find Difference

↓

Take Corrective Action

↓

Desired State Achieved

↓

Repeat Continuously

Example

Imperative: Docker

docker run nginx

"Start an Nginx container now.“

Declarative — Kubernetes

kubectl apply -f deployment.yaml

replicas: 3

"I want **3 Nginx Pods** running."

Kubernetes continuously monitors the cluster. If one Pod fails, Kubernetes detects the difference between the **desired state (3)** and **actual state (2)** and creates another Pod.

10Alytics | ACCREDITED BY THE AMERICAN COUNCIL OF TRAINING AND DEVELOPMENT

Key Features: What Makes Kubernetes Powerful

Kubernetes is more than a tool for starting containers. It provides a **production platform for automating, scaling, networking, and maintaining containerized applications**.

What Makes Kubernetes Powerful?

- **Self-Healing:** Automatically restarts failed Pods, replaces unhealthy Pods, and reschedules workloads when possible.
- **Horizontal Scaling:** Automatically increases or decreases Pod replicas based on CPU, memory, or custom metrics using the **Horizontal Pod Autoscaler (HPA)**.
- **Rolling Updates & Rollbacks:** Gradually deploy new application versions while maintaining availability, with the ability to roll back to a previous version.
- **Service Discovery & Load Balancing:** Kubernetes **Services** provide stable networking and distribute traffic across available Pods, while **CoreDNS** enables service discovery.

Storage Orchestration: Automatically provisions and mounts persistent storage through **PersistentVolumes (PV)** and **PersistentVolumeClaims (PVCs)**, including cloud and network storage.

Configuration & Secrets Management: **ConfigMaps** store application configuration, while **Secrets** manage sensitive values such as credentials and tokens separately from container images.

Key Takeaway

Kubernetes combines automation, resilience, scalability, networking, storage, and configuration management to keep applications running reliably at scale.

10Alytics | ACTD ACCREDITED BY THE AMERICAN COUNCIL OF TRAINING AND DEVELOPMENT

Kubernetes Use Cases — Where Is It Used?

Kubernetes has become a widely adopted platform for **running and managing containerized workloads at scale**. It is available across major cloud platforms and is used in many industries.

Where Is Kubernetes Used?

- **E-Commerce & Retail:** Automatically scale applications during traffic spikes, such as Black Friday or major sales events.
- **Financial Services:** Use **Namespaces, RBAC, Secrets, and resource controls** to isolate workloads and enforce access and security requirements.
- **Gaming & Streaming:** Dynamically run and scale game servers or streaming workloads based on demand.
- **AI & Machine Learning:** Schedule workloads across **GPU-enabled nodes for model training** and CPU-based resources for other workloads such as inference and APIs.
- **Healthcare & SaaS:** Support multi-tenant applications by isolating customers or workloads using **Namespaces, quotas, and access controls**.
- **CI/CD Pipelines:** Create temporary environments and test runners on demand, execute automated tests, and remove them when finished.

10Alytics

ACCREDITED BY THE AMERICAN COUNCIL OF TRAINING AND DEVELOPMENT

Kubernetes Use Cases — Where Is It Used?

Why Organizations Use Kubernetes

Kubernetes helps organizations **automate infrastructure management; scale applications based on demand, improve resource utilization, and deploy applications consistently**.

Key Takeaway

Kubernetes can run anything from a small application to large-scale, distributed workloads across multiple environments.

10Alytics
ACCREDITED BY THE AMERICAN COUNCIL OF TRAINING AND DEVELOPMENT

Kubernetes Pods: The Basic Deployment Unit

The smallest, most basic deployable object in Kubernetes

A **Pod** is the **smallest deployable unit in Kubernetes**. It represents one or more containers that are designed to run together as a single unit.

Think of a Pod as a **small workspace that holds one or more closely related containers**.

Key Features of a Pod

- **Shared Network:** Containers inside the same Pod share the **same IP address and network namespace**. They can communicate with each other using localhost.
- **Shared Storage:** Containers within a Pod can access the same **shared volumes**, allowing them to exchange files or data when required.
- **Co-Scheduled:** All containers in a Pod are scheduled onto the **same Worker Node** and run together throughout the Pod's lifecycle.
- **Sidecar Pattern:** A helper container can run alongside the main application to provide supporting functions such as **logging, monitoring, proxying, or configuration synchronization**.

Important Note

You **rarely create Pods directly** in production. Instead, Kubernetes controllers such as **Deployments, StatefulSets, and DaemonSets** create and manage Pods for you.

Key Takeaway: A Pod is the basic execution unit in Kubernetes. It provides a shared environment for one or more containers that need to run together.

10Alytics | ACCREDITED BY THE AMERICAN COUNCIL OF TRAINING AND DEVELOPMENT

Pod Lifecycle

Understanding pod phases: pod states, restart policies, and health checking

A Kubernetes Pod is not simply **"running" or "stopped."** Kubernetes tracks the Pod through defined **lifecycle phases** that help you understand its current state and troubleshoot problems.

Pod Phases

- **Pending:** The Pod has been accepted by Kubernetes but is not yet running. It may be waiting for scheduling, container image download, or other required resources.
- **Running:** The Pod has been successfully scheduled, and at least one container is running or starting.
- **Succeeded:** All containers in the Pod have completed successfully and exited with **exit code 0**.
- **Failed:** All containers have terminated, and at least one container exited with a **non-zero exit code** or failed to complete successfully.
- **Unknown:** Kubernetes cannot determine the Pod's current state, usually because it cannot communicate with the node where the Pod is running.

Why Pod Phases Matter

When troubleshooting, the Pod phase provides an **initial indication of what is happening**, but it is not enough by itself to identify the root cause.

Key Takeaway: Pod phases tell you the **high-level lifecycle state** of a Pod; use describe and logs to investigate *why* it is in that state.

Debug Commands

```
kubectl describe pod [pod-name]    # events + config
kubectl logs [pod-name]            # container stdout
kubectl logs [pod-name] -c sidecar # specific container
kubectl exec -it [pod-name] -- sh  # shell into pod
```

10Alytics | ACCREDITED BY THE AMERICAN COUNCIL OF TRAINING AND DEVELOPMENT

Pod Patterns: Working with Multiple Containers

Three proven patterns for running multiple containers in one Pod

Most Kubernetes Pods run **one main application container**. However, some applications benefit from additional containers that run alongside the main application and share the Pod's **network and storage**.

These supporting containers enable common architectural patterns.

Common Pod Patterns

- **Sidecar Pattern:** A helper container runs alongside the main application and provides supporting functionality such as **log collection, monitoring, proxying, or configuration synchronization**.
- **Ambassador Pattern:** A proxy container handles communication on behalf of the main application, such as **TLS termination, connection management, retries, or traffic routing**.
- **Adapter Pattern:** A helper container transforms or normalizes the main application's output into a format that another system can easily consume, for example, converting application logs into a standard format.

Init Container: Runs **before the main application containers start**. It performs initialization tasks such as checking dependencies, preparing configuration, downloading required files, or running database migrations.

Key Takeaway: Multiple containers in a Pod should be used when containers need to **share resources and work closely together**. Otherwise, keeping applications in separate Pods is usually simpler.

```
# Why use sidecar vs separate pod?
# Sidecar: tight coupling, shared localhost, shared volume
# Separate pod: independent lifecycle, scaling, deploys

# Check all containers in a pod:
kubectl get pod [pod-name] -o
jsonpath='{.spec.containers[*].name}'
```

10Alytics | ACCREDITED BY THE AMERICAN COUNCIL OF TRAINING AND DEVELOPMENT

Deployment: Managing Application Replicas

A **Deployment** is one of the most common Kubernetes resources for running **stateless applications**.

It manages a **ReplicaSet**, which in turn creates and maintains the required number of **Pods**.

Kubernetes continuously works to keep **3 Pods running**.

Key Features

- **Replica Management:** The ReplicaSet ensures the desired number of Pod replicas are running and replaces Pods when they fail.
- **Rolling Updates:** Deploy a new application version gradually by replacing old Pods with new ones, helping maintain service availability.
- **Rollback:** If a deployment causes problems, you can return to a previous revision.

kubectl rollout undo deployment/web

Revision History: Kubernetes maintains Deployment revisions, allowing you to inspect previous versions and deployment changes.

kubectl rollout history deployment/web

Key Takeaway: A **Deployment manages the desired state of your application**, while the **ReplicaSet maintains the required Pods** and supports updates and recovery.

```
# Create a deployment
kubectl create deploy web --image=nginx:1.25 --replicas=3

# Or apply from YAML (recommended)
kubectl apply -f deployment.yaml

# Watch rollout status
kubectl rollout status deploy/web
```

10Alytics | ACCREDITED BY THE AMERICAN COUNCIL OF TRAINING AND DEVELOPMENT

Deployment YAML: The Application Blueprint

A **Deployment YAML manifest** describes the desired state of your application. It acts as the **configuration blueprint** Kubernetes uses to create, update, and manage your Pods.

Understanding the key fields helps you confidently manage Kubernetes workloads.

Important Deployment Fields

- **replicas: 3 —** Defines the desired number of Pod replicas Kubernetes should maintain.
- **selector.matchLabels —** Defines which Pods belong to and are managed by the Deployment.
- **template.spec.containers —** Defines the containers to run, including the **image, ports, environment variables, and resource settings**.
- **resources.requests —** Specifies the CPU and memory resources a container requests. Kubernetes uses these values when scheduling Pods.
- **resources.limits —** Defines the maximum CPU and memory resources a container can consume.

Rolling Update Controls

maxUnavailable: 1-Maximum number of Pods that can be unavailable during a rolling update.

maxSurge: 1-Maximum number of additional Pods that can temporarily run above the desired replica count during an update.

Key Takeaway: The Deployment manifest defines **what you want**, while Kubernetes uses that configuration to **create, scale, update, and maintain the application**.

Common Deployment Commands

Scale the application:

kubectl scale deployment/web --replicas=10

Changes the desired number of replicas to **10**.

Update the container image:

kubectl set image deployment/web web=nginx:1.26

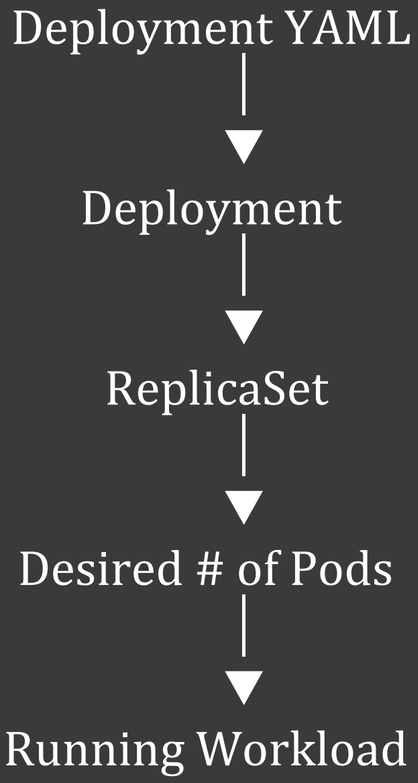
Changes the application's container image and triggers a rollout.

Rollback to a previous revision:

kubectl rollout undo deployment/web --to-revision=2

Restores the Deployment to **revision 2**.

How the Pieces Connect



10Alytics

ACCREDITED BY THE AMERICAN COUNCIL OF TRAINING AND DEVELOPMENT

SESSION CHECK-IN ✓

Attendance check-in

Mark yourself present

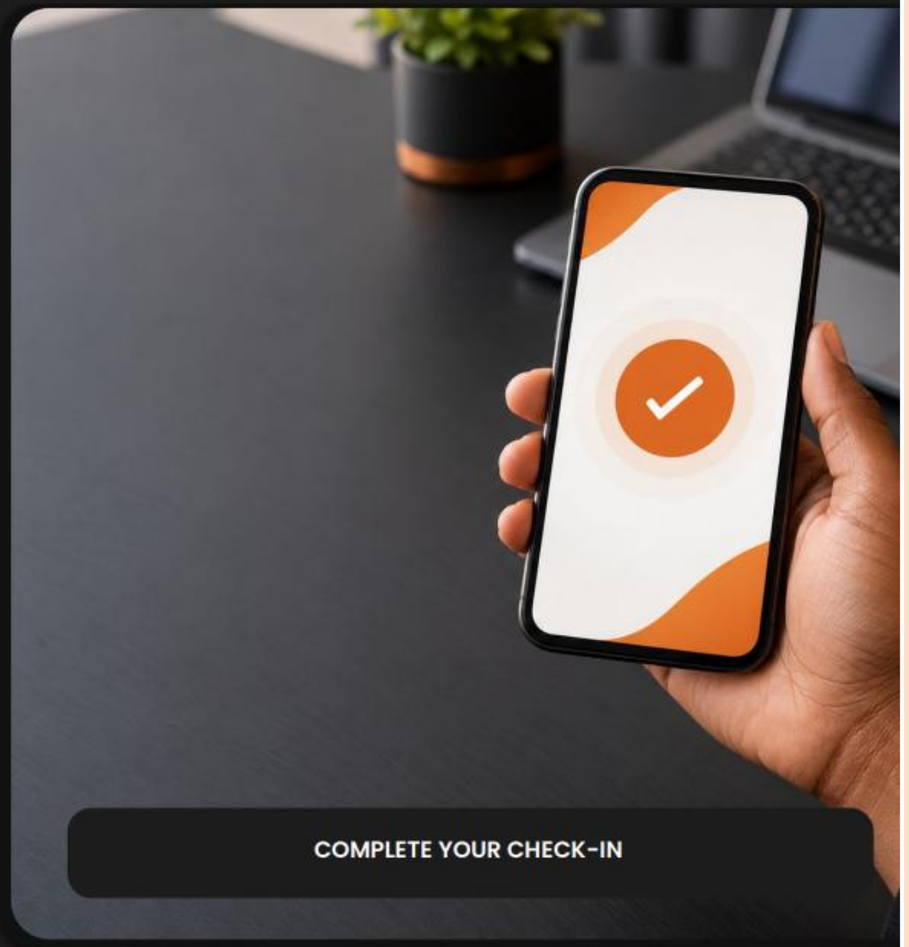
Open the attendance link and complete the form once.

CLICK HERE

Use the link above to record your attendance.

FACILITATOR	TOPIC
[Name]	[Title]

Thank you for checking in.



10Alytics | ACCREDITED BY THE AMERICAN COUNCIL OF TRAINING AND DEVELOPMENT

Kubernetes Rolling Update

What Is a Rolling Update?

Rolling Update is the default Deployment strategy in Kubernetes.

Instead of stopping all existing Pods and starting new ones at once, Kubernetes **gradually replaces old Pods with new Pods**.

This helps keep the application **available throughout the update process**.

🔄 Rolling Update Process

Create a New Pod

- Kubernetes creates a new Pod using the updated container image.

Check Readiness

- Kubernetes waits for the new Pod to pass its **readinessProbe**.
- A Pod that is not ready does **not** receive application traffic.

Remove an Old Pod from Service

- Once the new Pod is ready, Kubernetes removes an old Pod from the Service's available endpoints.
- Traffic is no longer sent to that Pod.

Gracefully Terminate the Old Pod

- Kubernetes sends **SIGTERM** to allow the application to shut down cleanly.
- It waits for the configured termination period.
- If the container does not stop, Kubernetes eventually sends **SIGKILL**.

Repeat the Process

- Kubernetes continues creating new Pods and removing old Pods until **all Pods are running the new version**.

10Alytics | ACCREDITED BY THE AMERICAN COUNCIL OF TRAINING AND DEVELOPMENT

Why Readiness Matters

🚦 Why Readiness Matters

A Kubernetes **Service** sends traffic only to Pods that are considered **Ready**.

Therefore:

New Pod → Readiness Check → Ready ✓ → Receives Traffic

If the new Pod fails its readiness check, Kubernetes does not send normal Service traffic to it.

Key Takeaway

Rolling Updates allow Kubernetes to deploy a new application version gradually while keeping the existing service available.

In simple terms:

Create → Test → Send Traffic → Remove Old → Repeat

```
# Update image
kubectl set image deploy/web web=myapp:v2.0.0

# Watch it happen in real-time
kubectl rollout status deploy/web

# Something wrong? Instant rollback!
kubectl rollout undo deploy/web

# Go back to a specific version
kubectl rollout undo deploy/web --to-revision=3
```

10Alytics | ACTD ACCREDITED BY THE AMERICAN COUNCIL OF TRAINING AND DEVELOPMENT

Kubernetes Service

What Is a Service?

Kubernetes **Pods are ephemeral**, meaning they can be created, destroyed, restarted, or rescheduled at any time.

Because Pods can receive **new IP addresses**, connecting directly to Pod IPs is unreliable.

A **Service** provides a **stable virtual IP address and DNS name** that routes traffic to the appropriate healthy Pods.

🌐 What Does a Service Provide?

1. Stable Endpoint

- The Service gets a stable **ClusterIP** and DNS name.
- The Service address remains available even when Pods restart or are replaced.
- Applications do not need to know individual Pod IP addresses.

2. Service Discovery

- Other Pods can access a Service using its **DNS name** instead of a hardcoded IP address.

3. Load Balancing

- The Service distributes requests across matching **Ready/healthy endpoints**.
- This allows multiple Pods to serve the same application.
- Kubernetes provides load balancing across the Service's endpoints; the exact distribution behavior depends on the networking implementation rather than guaranteeing strict round-robin.

10Alytics | ACCREDITED BY THE AMERICAN COUNCIL OF TRAINING AND DEVELOPMENT

Kubernetes Service

4. Label Selector

- A Service uses a **label selector** to identify which Pods should receive traffic.
- New Pods with matching labels are automatically included as Service endpoints when they become ready.

Key Takeaway

A Service provides a stable way to access dynamic Pods.

Remember:

Pods change → Service stays stable → Traffic continues

```
Create a Service for a Deployment
kubectl expose deploy/web --port=80 --target-port=8080

# Or apply from YAML
kubectl apply -f service.yaml

# See all services and their IPs
kubectl get svc

# Test from inside cluster
kubectl run test --rm -it --image=busybox -- wget -q
http://web-svc
```

10Alytics | ACCREDITED BY THE AMERICAN COUNCIL OF TRAINING AND DEVELOPMENT

Service Types

Kubernetes provides different ways to expose applications.

Each option serves a different **network traffic pattern**.

Choosing the appropriate exposure method helps achieve the required **accessibility, security, and scalability**.

Main Service Types

1. ClusterIP — Internal Access

- **ClusterIP is the default Service type.**
- Accessible **only within the Kubernetes cluster**.
- Used when applications need to communicate internally.

Common examples:

1. Databases
2. Redis/cache services
3. Internal APIs
4. Backend services

2. NodePort — Basic External Access

NodePort exposes a Service through a port on each Kubernetes node.

- External clients access the application using:

NodeIP:NodePort

- Useful for **development, testing, and simple environments**.
- Typically exposes a port from the NodePort range **30000–32767** by default.

Internet

▼

NodeIP:30080

▼

Service

▼

Application Pods

Use when: You need simple external access without a dedicated external load balancer.

10Alytics | ACTD ACCREDITED BY THE AMERICAN COUNCIL OF TRAINING AND DEVELOPMENT

Service Types

3. LoadBalancer — External Load Balancing

LoadBalancer exposes a Service through an external load balancer, typically provided by a cloud or infrastructure platform.

Examples include:

- AWS Elastic Load Balancing
- Google Cloud Load Balancing
- Azure Load Balancer

Use when: A service needs direct external access through an external load-balancing solution.

4. Ingress — HTTP/HTTPS Routing

Ingress provides Layer 7 HTTP/HTTPS routing into Kubernetes.

Instead of exposing every application through a separate external IP or port, an Ingress can route traffic to multiple Services based on:

- **Hostname**
- **URL path**
- **TLS/HTTPS configuration**

Example routing:

- *api.example.com/users → User Service*
- *example.com/shop → Shop Service*

Ingress commonly handles **TLS termination**, allowing HTTPS traffic to be decrypted at the ingress layer before being forwarded to backend Services.

Important: Ingress is not itself a Service type. It is a separate Kubernetes API resource that works with Services and requires an **Ingress Controller** to implement the routing.

```
# service.yaml
spec:
  type: LoadBalancer     # ClusterIP | NodePort | LoadBalancer
  selector:
    app: webserver       # Routes to pods with this label
  ports:
  - port: 80             # Port the Service listens on
    targetPort: 8080     # Port the Pod listens on
```

Quick Comparison

Type	Access	Typical Use
ClusterIP	Internal	Backend, database, cache
NodePort	External via Node IP + port	Development/testing
LoadBalancer	External Load Balancer	Public services
Ingress	HTTP/HTTPS routing	Multiple web services, domains, paths

Key Takeaway

- **ClusterIP = Internal access**
- **NodePort = Node IP + Port**
- **LoadBalancer = External Load Balancer**
- **Ingress = Smart HTTP/HTTPS Routing**

Remember:

Service exposes your application; Ingress intelligently routes web traffic to multiple Services.

10Alytics | ACCREDITED BY THE AMERICAN COUNCIL OF TRAINING AND DEVELOPMENT

Ingress

What Is an Ingress?

An **Ingress** is a **Layer 7 (HTTP/HTTPS) routing mechanism** in Kubernetes.

Instead of exposing every application through a separate external LoadBalancer, an Ingress can provide **one entry point** and route incoming web traffic to different Services based on the **hostname or URL path**.

Key Capabilities

1. Cost Efficient: This can reduce the number of external load-balancing resources required, although the actual cost depends on the cloud provider and architecture.
2. **Hostname-Based Routing:** Traffic can be routed based on the requested domain.
3. **Path-Based Routing:** Traffic can also be routed based on the URL path.

4. TLS Termination

Ingress can handle **HTTPS/TLS encryption** at the entry point.

The TLS certificate can be managed at the Ingress layer, allowing backend Pods to receive HTTP traffic when appropriate for the internal network and security design.

Note: Using plain HTTP internally is a design choice, not a requirement. Sensitive environments may use encryption all the way to the Pods.

```
spec:
  rules:
  - host: api.myapp.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: api-service
            port: { number: 80 }
```

Ingress Controller

Popular Ingress Controllers

An **Ingress resource only defines the routing rules.** An **Ingress Controller** is responsible for actually implementing those rules.

Popular options include:

- **NGINX Ingress Controller**
- **Traefik**
- **AWS Load Balancer Controller**
- **Kong Gateway**

How to Register Your Pod

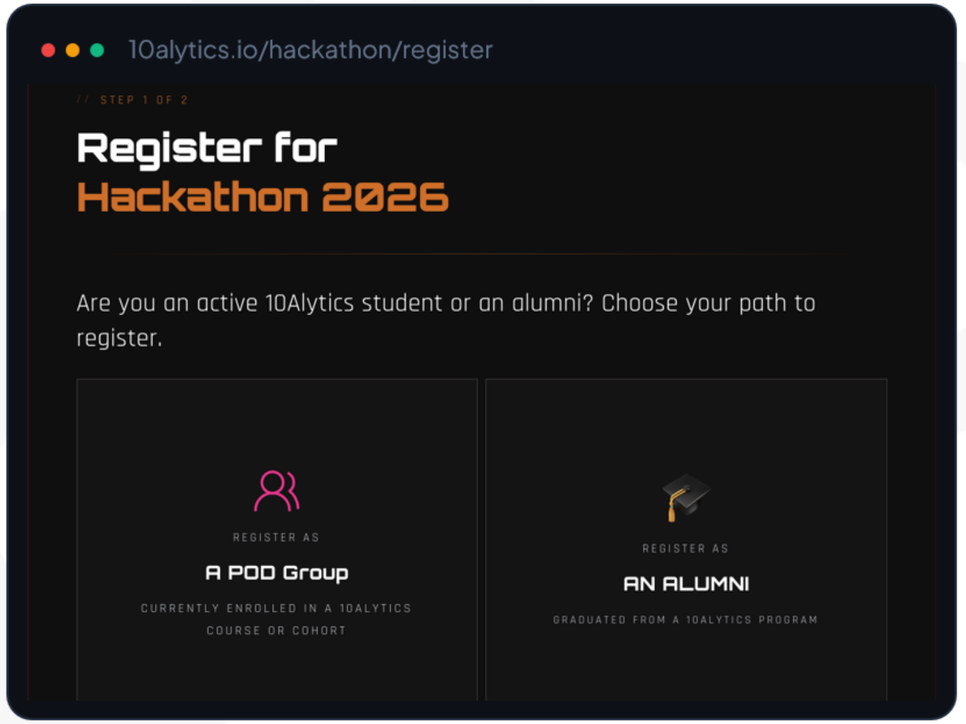
10Alytics

ACCREDITED BY THE AMERICAN COUNCIL OF TRAINING AND DEVELOPMENT

01 **Visit Registration Portal**
Visit page: https://www.10alytics.io/hackathon/register

02 **Select "A POD Group"**
Any member can submit on behalf of the pod

03 **Authenticate & Confirm**
Log in with credentials and confirm pod email



Only one submission required per pod team

10Alytics Student's Hackathon Registration

10Alytics

ACCREDITED BY THE
AMERICAN COUNCIL OF
TRAINING AND DEVELOPMENT

SESSION PAUSE ⏸

Coffee break

15 minutes to recharge

Stretch, refill and take a breather. We'll resume promptly.

WE'LL RESUME AT

[TIME]

Please return on time so we can restart together.

